

# Reviewer Pack — Minimal Rank of Formal Contexts and Resolution Proof Width L...

Entry bed1075d7871 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 10:26:08 UTC

## Minimal Rank of Formal Contexts and Resolution Proof Width Lower Bound

**Entry ID:** bed1075d7871 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 10:26:08 UTC

### 1. Conjecture

**Field A** (mathematical branch): Lattice Theory (Formal Contexts) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

#### Statement:

The minimal rank of the formal context associated with a given CNF  $\varphi$  is  $O(\log w(\varphi))$ , where  $w(\varphi)$  is the width of the resolution proof for  $\varphi$ .

#### Rationale (proposer's reasoning):

Formal contexts provide a way to represent Boolean functions in terms of their lattice structure, which can offer insights into computational complexity. By examining the rank of these formal contexts, we may uncover structural properties that correlate with the difficulty of finding a resolution proof for a CNF.

**Taxonomy category:** FormalContextsResolutionProofComplexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** b101feb28633d4dc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if and only if the correlation coefficient between  $\min\_rank(C(\varphi))$  and  $\log(w(\varphi))$  exceeds 0.9 for at least 80% of the seeds, with no seed's  $\min\_rank(C(\varphi))$  exceeding  $\log(w(\varphi))$  by more than 5 units.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 1.00       | SAFE             | UNCERTAIN        |

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** ADJACENT\_OK against 3 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - "formal contexts" AND "resolution proof complexity" AND minimal rank"  
- "CNF formal context" AND "width resolution proof" - "Boolean satisfiability" AND lattice theory AND proof lower bound

**Top relevant hits considered:** - [http://arxiv.org/abs/1510.02682v8] Classical and Quantum Algorithms for the Boolean Satisfiability Problem - [http://arxiv.org/abs/1910.10708v5] Necessary and sufficient conditions for Boolean satisfiability - [http://arxiv.org/abs/2011.06551v1] Efficient Solution of Boolean Satisfiability Problems with Digital MemComputing

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def dpll_solve(cnf):
        n = max(abs(lit) for clause in cnf for lit in clause)
        literals = [i for i in range(1, n + 1)] + [-i for i in range(1, n + 1)]

        def is_satisfiable(model):
            for clause in cnf:
                if all(lit not in model or model[lit] != (lit > 0) for lit in clause):
                    return False
            return True

        def backtrack(model, literals):
            if len(model) == n:
                return is_satisfiable(model)
            literal = literals[0]
            if backtrack(model | {literal: True}, literals[1:]):
                return True
            if backtrack(model | {literal: False}, literals[1:]):
                return True
            return False

        return backtrack({}, literals)

    def formal_context(cnf):
```

```

minterms = set()
non_minterms = set()
for clause in cnf:
    if len(clause) == 1:
        minterms.add(clause[0])
    else:
        non_minterms.add(tuple(sorted(clause)))

universe = list(minterms) + list(non_minterms)
R = {}
for x in universe:
    R[x] = set()

for clause in cnf:
    if len(clause) == 1:
        continue
    for i in range(len(clause)):
        for j in range(i + 1, len(clause)):
            R[clause[i]].add(clause[j])
            R[clause[j]].add(clause[i])

return universe, R

def min_rank(universe, R):
    n = len(universe)
    rank = [0] * n
    visited = [False] * n

    for i in range(n):
        if not visited[i]:
            queue = [i]
            while queue:
                u = queue.pop(0)
                if not visited[u]:
                    visited[u] = True
                    for v in R[universe[u]]:
                        rank[v] += 1
                        queue.append(v)

    return max(rank)

def generate_cnf(n, m):
    cnf = []
    literals = list(range(1, n + 1)) + [-i for i in range(1, n + 1)]
    random.shuffle(literals)
    for _ in range(m):
        clause = [literals.pop() for _ in range(random.randint(2, 3))]
        cnf.append(clause)
    return cnf

def width(cnf):
    clauses = set(tuple(sorted(clause)) for clause in cnf)
    max_width = 0
    while clauses:
        unit_clauses = [clause for clause in clauses if len(clause) == 1]
        if not unit_clauses:

```

```

        break
    literal = unit_clauses[0][0]
    new_clauses = set()
    for clause in clauses:
        if literal not in clause and -literal not in clause:
            new_clauses.add(tuple(sorted(set(clause) - {literal, -literal})))
    max_width += 1
    clauses = new_clauses
    return max_width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5):
        cnf = generate_cnf(n, random.randint(2 * n, 3 * n))
        if not dpll_solve(cnf):
            continue
        universe, R = formal_context(cnf)
        min_rank_value = min_rank(universe, R)
        width_value = width(cnf)
        results.append((min_rank_value, math.log(width_value)))

if len(results) < 30:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

mean_metric = sum(value for _, value in results) / len(results)
std_metric = math.sqrt(sum((value - mean_metric) ** 2 for _, value in results) / len(results))
correlation_coefficient = sum((min_rank_value - mean_metric) * (math.log(width_value) - mean_log_width)) / len(results)

return {
    "metric_name": "correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient > 0.9 and all(abs(min_rank_value - math.log(width_value)) < 0.1 for _, value in results),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3**j + 5**k for i in range(10) for j in range(10) for k in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result["metric_value"])

    mean_metric = sum(results) / len(results)

```

```

std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in results) / len(results))
support_fraction = sum(1 for x in results if x is not None and x > 0.9) / len(results)

if all(x is not None and x > 0.9 for x in results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
elif any(x is not None and x <= 0.9 for x in results) or any(abs(x - math.log(y)) > 5 for x, y
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient\" first_failing_seed={s
else:
    print("RESULT: INCONCLUSIVE not_enough_data")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faca7f47.py", line 151, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faca7f47.py", line 115, in run_trial
    cnf = generate_cnf(n, random.randint(2 * n, 3 * n))
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faca7f47.py", line 90, in generate_cnf
    clause = [literals.pop() for _ in range(random.randint(2, 3))]
              ~~~~~^
IndexError: pop from empty list

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which means it was unable to complete the required computations to verify the conjecture. | next: Investigate and fix the crash in the test code to allow for proper verification of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model       | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|-------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest | 0         | 0   | 14256        |
| 2 | preregistration | ollama_remote | glm4:latest | 0         | 0   | 10706        |
| 3 | novelty         | ollama_remote | glm4:latest | 0         | 0   | 8405         |
| 4 | novelty         | ollama_remote | glm4:latest | 0         | 0   | 16337        |

| # | Phase    | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|----------|---------------|------------------|-----------|-----|--------------|
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 25488        |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14548        |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17749        |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17038        |
| 9 | judge    | ollama_remote | glm4:latest      | 0         | 0   | 15466        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 139993 ms total latency. Provider mix: {'ollama\_remote': 9}

*(full prompt+response transcripts available in research/audit/bed1075d7871.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bed1075d7871.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/bed1075d7871.tar.gz (if generated)